

Formulário HTML

Nome completo:

Idade:

Profissão:

Salário:

Experiência:

Sem experiencia

Validações

Nome completo:

Idade:

Profissão:

Salário:

Experiência:

Sem experiencia

Nome completo:

Idade:

Profissão:

Salário:

Experiência:

 Preencha este campo.

Retorno PHP

Nome completo: Rafaela

Idade: 26

Profissão: Estudante

Salário: R\$ 0

Experiência: Sem experiência

Obrigado pelo envio das suas informações, Rafaela!

Ficamos a disposição para auxiliá-lo a melhorar a sua experiência (Sem experiência) referente a sua atual profissão: Estudante

[Voltar ao Formulário](#)

Prompt utilizado:

gem, estou criando um formulario em html e estou solicitando o nome do usuario, gostaria de alguma forma de travar o campo para aceitar apenas letras e nao numeros, como faço?

A maneira mais eficiente de impedir números em um campo de nome é combinar o bloqueio em tempo real via JavaScript com o atributo de validação nativo `pattern` do HTML5.

1. Bloqueio em tempo real (JavaScript no input)

Remove números e símbolos instantaneamente à medida que o usuário digita:

HTML



```
<label for="nome">Nome completo:</label>
<input
  type="text"
  id="nome"
  name="nome"
  placeholder="Digite seu nome"
  oninput="this.value = this.value.replace(/^[^a-zA-ZÀ-ÿ\s]/g, '')"
  required
>
```

2. Validação no envio do formulário (Atributo `pattern`)

Impede a submissão do formulário caso o campo contenha caracteres inválidos, exibindo um alerta nativo do navegador:

HTML



```
<label for="nome">Nome completo:</label>
<input
  type="text"
```

+ Peça ao Gemini

Flash ▾



HTML

```
type="text"
id="nome"
name="nome"
pattern="[A-Za-zÀ-ÿ\s]+"
title="Digite apenas letras e espaços."
required
>
```

Entendendo a Expressão Regular (Regex):

- `a-zA-Z` : Permite letras de A a Z (maiúsculas e minúsculas).
- `À-ÿ` : Permite acentos e caracteres como ç, ã, é, ô.
- `\s` : Permite espaços para aceitar sobrenomes.
- `^` : Inverte a regra (no JavaScript, apaga tudo o que não for o padrão definido).

3. Abordagem recomendada com JavaScript separado

Para manter o HTML limpo e organizar melhor o código:

HTML

```
<input type="text" id="campo-nome" placeholder="Digite seu nome">

<script>
  const campoNome = document.getElementById('campo-nome');

  campoNome.addEventListener('input', (event) => {
    event.target.value = event.target.value.replace(/^[a-zA-ZÀ-ÿ\s]/g, '');
  });
</script>
```

Quer ver como fazer essa mesma validação de nome no lado do servidor (backend)?

+ Peça ao Gemini

Flash



Do retorno do prompt foi utilizado o pattern para evitar que o usuário insira números no nome.

ótimo, e como faço um campo de texto longo

Para campos de texto longo (como mensagens, biografias ou comentários), utilize a tag `<textarea>`. Diferente do `<input>`, ela aceita múltiplas linhas e quebras de texto.

HTML

```
<label for="mensagem">Sua mensagem:</label>
<textarea
  id="mensagem"
  name="mensagem"
  rows="5"
  placeholder="Escreva seu texto aqui..."
  maxlength="500"
  required
></textarea>
```

Principais atributos do `<textarea>`:

- `rows`: Define a altura inicial do campo em linhas de texto.
- `maxlength`: Limita o total máximo de caracteres permitidos.
- `placeholder`: Exibe uma dica de preenchimento temporária.

Dica de CSS:

Por padrão, o navegador permite que o usuário redimensione a caixa para os lados, o que pode quebrar o layout da página. Para evitar isso, controle a propriedade `resize`:

CSS

```
textarea {
  width: 100%;
  resize: vertical; /* Permite redimensionar apenas a altura, nunca a largura */
  /* Use "resize: none;" para travar o tamanho completamente */
}
```

+ Peça ao Gemini

Flash ▾



O Gemini é uma IA e pode cometer erros.

Referente ao retorno do campo texto, foi utilizado a tag `textarea`, `rows` e `maxlength` para criar o campo texto para a experiência do usuário.

```
incluir uma última linha no php e simplesmente não aparece  
no código<?php  
echo "Nome completo: ".$_POST['nome']."<br>";  
echo "Idade: ".$_POST['idade']."<br>";  
echo "Profissão: ".$_POST['profi']."<br>";  
echo "Salário: R$ ".$_POST['sal']."<br>";  
echo "Experiência: ".$_POST['exp']."<br>";  
echo "Obrigado pelo envio das suas informações,  
".$_POST['nome']."!";  
?>
```

O código está correto! Se a nova linha não está aparecendo na tela quando você envia o formulário, isso geralmente acontece por **cache do navegador** ou por ter esquecido de **salvar o arquivo** após a alteração.

Siga estas verificações rápidas para resolver:

1. Salve o arquivo no editor

- Certifique-se de salvar o arquivo `PHP_FORMULARIO.php` no seu editor de código (pressione `Ctrl + S` no VS Code, Notepad++ ou Bloco de Notas).

2. Force a atualização da página no navegador (Limpar Cache)

- Volte na aba do navegador onde o formulário está aberto (`http://localhost/Agenda02_Rafaela/HTML_Formulario.html`).
- Pressione `Ctrl + F5` (ou `Ctrl + Shift + R`) para recarregar a página limpando o cache.
- Preencha os dados e clique em enviar novamente.

3. Validação do campo `nome`

+ Peça ao Gemini

Flash ▾



cache.

- Preencha os dados e clique em enviar novamente.

3. Validação do campo `nome`

- Caso o campo `nome` no HTML não esteja preenchido, a string ficará assim:
"Obrigado pelo envio das suas informações, !" . Verifique se o atributo `name="nome"` no HTML corresponde exatamente ao que está no PHP.

Dica para evitar erros de campos em branco no PHP

Se algum campo não for preenchido, o PHP pode emitir um aviso (*Warning: Undefined array key*). Para deixar seu código mais seguro:

PHP

```
<?php
$nome = $_POST['nome'] ?? '';
$idade = $_POST['idade'] ?? '';
$profi = $_POST['profi'] ?? '';
$sal = $_POST['sal'] ?? '';
$exp = $_POST['exp'] ?? '';

echo "Nome completo: " . $nome . "<br>";
echo "Idade: " . $idade . "<br>";
echo "Profissão: " . $profi . "<br>";
echo "Salário: R$ " . $sal . "<br>";
echo "Experiência: " . $exp . "<br>";
echo "Obrigado pelo envio das suas informações, " . $nome . "!";
?>
```

🔖 🗨️ 📄 ...

+ Peça ao Gemini

Flash ▾



O Gemini é uma IA e pode cometer erros.

Validação sobre a linha de código inserida em PHP

```
 vamos por partes o que isso faz e por que em "??" em cada
 variavel? $nome = $_POST['nome'] ?? ";
 $idade = $_POST['idade'] ?? ";
 $profi = $_POST['profi'] ?? ";
 $sal = $_POST['sal'] ?? ";
 $exp = $_POST['exp'] ?? ";
```

A sintaxe `?? ''` é chamada de **Operador de Coalescência Nula** (*Null Coalescing Operator*).

Ela serve para atribuir um valor padrão de segurança caso a informação buscada no formulário não exista ou seja nula.

O que o código faz linha por linha:

1. `$_POST['nome']` : Tenta capturar o valor digitado no campo `name="nome"` do seu formulário HTML.
2. `??` : Funciona como uma pergunta: *"Esse campo foi enviado na requisição e possui algum valor?"*
3. `''` : É o valor padrão (uma string vazia). Se a resposta para a pergunta do `??` for "não", o PHP usa esse texto vazio.
4. `$nome = ...` : Armazena o resultado final na variável.

Por que usamos isso?

Se você tentasse usar apenas `$nome = $_POST['nome'];` e o usuário acessasse a página diretamente (sem passar pelo formulário) ou se algum campo do HTML não enviasse o parâmetro, o PHP geraria um erro de aviso:

```
Warning: Undefined array key "nome" in ...
```

O operador `?? ''` impede que esse erro aconteça.

Resumo do comportamento:

- **Se o campo veio do formulário:** A variável guarda o que o usuário digitou (ex: `"Rafaela"`).
- **Se o campo NÃO veio ou veio nulo:** A variável guarda `''` (texto vazio) em vez de travar o código com um erro.

Quer ver como validar se esses campos vieram vazios antes de exibir o texto?

+ Peça ao Gemini

Flash ▾



O Gemini é uma IA e pode cometer erros.

Solicitada explicação sobre os campos "??" e os mesmos foram agregados ao código para armazenamento das informações dos usuários.


```
analise o código php e indique melhorias necessárias: <?php
$nome = $_POST['nome'] ?? '';
$idade = $_POST['idade'] ?? '';
$profi = $_POST['profissao'] ?? '';
```

O seu código já está funcionando e com a sintaxe correta do PHP, mas existem 4 pontos críticos de melhoria em relação a boas práticas e segurança:

- **Uso das variáveis declaradas:** Você definiu variáveis no topo com a proteção `?? ''` (`$nome`, `$profi`, etc.), mas nos comandos `echo` continuou chamando `$_POST['nome']` diretamente. Se algum campo vier vazio, o aviso de erro do PHP ainda pode acontecer.
- **Segurança contra ataques XSS:** Exibir dados digitados pelo usuário diretamente na tela via `echo` abre margem para scripts maliciosos. O recomendado é tratar os valores com a função `htmlspecialchars()`.
- **Inconsistência nos nomes das chaves:** Verifique se no arquivo HTML o atributo `name` dos inputs mudou para `profissao` e `salario` (nos exemplos anteriores estava `profi` e `sal`). Os nomes precisam ser idênticos em ambos os arquivos.
- **Estrutura e Acentuação:** Sem a declaração do `<meta charset="UTF-8">`, palavras com acento como *"disposição"* e *"auxiliá-lo"* podem aparecer distorcidas no navegador.

Código Refatorado e Otimizado

PHP



```
<?php
// Captura os dados e aplica sanitização contra XSS
$nome      = htmlspecialchars($_POST['nome'] ?? '');
$idade     = htmlspecialchars($_POST['idade'] ?? '');
$profissao = htmlspecialchars($_POST['profissao'] ?? '');
$salario   = htmlspecialchars($_POST['salario'] ?? '');
$exp       = htmlspecialchars($_POST['exp'] ?? '');
?>

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Confirmação de Envio</title>
```

+ Peça ao Gemini

Flash ▾



PHP



```
$nome      = htmlspecialchars($_POST['nome'] ?? '');
$idade     = htmlspecialchars($_POST['idade'] ?? '');
$profissao = htmlspecialchars($_POST['profissao'] ?? '');
$salario   = htmlspecialchars($_POST['salario'] ?? '');
$exp       = htmlspecialchars($_POST['exp'] ?? '');
?>

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Confirmação de Envio</title>
</head>
<body>

    <h2>Dados Cadastrados</h2>

    <p>
        <strong>Nome completo:</strong> <?= $nome ?><br>
        <strong>Idade:</strong> <?= $idade ?><br>
        <strong>Profissão:</strong> <?= $profissao ?><br>
        <strong>Salário:</strong> R$ <?= $salario ?><br>
        <strong>Experiência:</strong> <?= $exp ?>
    </p>

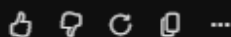
    <p>
        Obrigado pelo envio das suas informações, <strong><?= $nome ?></strong>!<br>
        Ficamos à disposição para auxiliá-lo a melhorar a sua experiência (<?= $exp
    </p>

    <a href="HTML_Formulario.html">
        <button type="button">Voltar ao Formulário</button>
    </a>

</body>
</html>
```

Quer ver como formatar o valor do salário para o padrão monetário brasileiro (ex: 2.500,00) com PHP?

↩ Sim



+ Peça ao Gemini

Flash ▾



O Gemini é uma IA e pode cometer erros.